

Laboratorio IA 8: Redes con memoria sensorial para romper ciclos en navegacion reactiva

Materia: Inteligencia Artificial

Dinamica: practica en terminal **Entregable:** codigo, resultados, analisis y conclusion

Extender la base de la red neuronal evolutiva para que el robot recuerde su estado reciente y reduzca la probabilidad de quedar atrapado en ciclos. La tarea consiste en revisar la estructura de los laboratorios previos y modificar la implementacion entregada, manteniendo la misma logica general del robot, el fitness y la evolucion genetica.

1. Continuidad con las clases anteriores

El laboratorio conserva el mundo del robot usado en las sesiones anteriores: una cuadrícula, un punto de inicio, una meta, obstaculos, sensores locales y una funcion de fitness. La progresion es:

- En las primeras clases se evoluciono una secuencia de movimientos.
- Despues se incorporaron obstaculos y se compararon mutacion y cruce.
- Con la percepcion local, el robot paso a consultar el entorno antes de decidir.
- En el laboratorio 7 se reemplazo la tabla de reglas por una red neuronal.
- En este laboratorio se conserva esa base, pero se agrega memoria temporal para que el robot recuerde su estado anterior y la ultima accion.

La evolucion sigue seleccionando comportamientos por fitness. Lo que cambia es la representacion del estado de entrada: antes la red solo observaba el presente; ahora ademas incorpora lo visto y hecho en el instante anterior.

La idea original del laboratorio anterior era:

$$a_t = \pi(s_t)$$

donde s_t representa una descripcion local del entorno en el instante t .

Eso ya funcionaba como una politica reactiva, pero tenia un problema: el robot no recordaba lo que habia visto un paso antes ni lo que habia decidido antes. Asin, podia entrar en secuencias repetitivas que parecian razonables localmente pero que no avanzaban al objetivo.

No se trata de reescribir el robot desde cero. La tarea es identificar que parte del codigo anterior se conserva y que parte debe cambiar para incluir memoria temporal y evitar ciclos.

2. El ciclo sensorimotor

El comportamiento de un agente artificial puede describirse como un ciclo:

$$\text{entorno} \rightarrow \text{sensores} \rightarrow \text{red} \rightarrow \text{accion} \rightarrow \text{entorno}$$

Este ciclo se repite para cada instante $t, t+1, t+2, \dots$.

En este contexto, se habla de ciclo sensorimotor: una secuencia de percepcion, accion, percepcion, accion. Cuando la observacion vuelve a un estado similar y la accion anterior reaparece, el agente puede quedar atrapado en un bucle.

3. Estado sensorial anterior y accion previa

La red del laboratorio anterior recibia solo:

$$s_t = [U_t, D_t, L_t, R_t]$$

Ahora, la entrada se amplia con la informacion que ya existe en el flujo del robot: la observacion del paso anterior y la accion que se ejecuto antes. Es decir, la politica pasa a depender de:

$$a_t = \pi(s_t, s_{t-1}, a_{t-1})$$

donde s_t es la percepcion actual, s_{t-1} la percepcion anterior y a_{t-1} la ultima accion ejecutada.

La entrada total queda formada por 12 valores:

- 4 sensores actuales,
- 4 sensores anteriores,
- 4 bits de la accion previa (codificacion one-hot).

Esto funciona como una ventana temporal de corto alcance. El robot ya no decide solo con el presente, sino con una memoria reciente del estado.

4. La red neuronal

La arquitectura sigue siendo sencilla pero ahora con entrada ampliada:

Entrada	Capa oculta	Salida
12 valores temporales $[s_t, s_{t-1}, a_{t-1}]$	16 neuronas con ReLU combinacion no lineal	4 acciones un valor por accion

La salida no es una accion directamente. Son cuatro valores llamados logits. La accion elegida es la que tenga el valor maximo:

```
logits = red(entrada)[0]
indice = int(torch.argmax(logits).item())
action = COMANDOS[indice]
```

Cada individuo de la poblacion sigue siendo una instancia distinta de la clase de red, pero ahora el vector de entrada incluye memoria sensorial. Sus pesos siguen funcionando como material genetico.

$$s_{t-1} = [U_{t-1}, D_{t-1}, L_{t-1}, R_{t-1}]$$

$$a_{t-1}$$

Es decir, la politica se vuelve:

$$a_t = \pi(s_t, s_{t-1}, a_{t-1})$$

Esto significa que la red no decide solo con la situacion presente, sino con un estado reciente y con la historia inmediata de sus decisiones.

5. Fitness y algoritmo evolutivo

Cada red recorre el mapa durante como maximo 30 pasos. El fitness puede mantenerse similar al laboratorio anterior y, si se desea, incluir una penalizacion extra por ciclos o repeticiones. La idea es conservar la estructura del problema y solo ajustar la politica y la evaluacion:

$$F = 500 - 35d + 4p - 30c - 3r + 2000m - 10b$$

Donde d es la distancia Manhattan a la meta, p los pasos utiles, c los choques, r las posiciones repetidas, m vale 1 cuando se alcanza la meta y b penaliza la aparicion de bucles. En cada generacion:

1. Se evalua toda la poblacion.
2. Se ordenan los individuos por fitness.
3. Se conservan seis elites.
4. Se seleccionan padres entre los mejores individuos.
5. Se aplica cruce en un punto y mutacion gaussiana sobre los pesos.
6. Se completa la siguiente poblacion hasta llegar a 40 individuos.

Por defecto se ejecutan 40 generaciones y la semilla es aleatoria. Se puede fijar una semilla para reproducir un experimento concreto.

6. Ejecutar el laboratorio

Ejecucion normal, con 40 generaciones, semilla aleatoria y animacion:

```
python3 robot_red_neuronal_memoria.py
```

Ejecucion reproducible y sin pausa entre cuadros:

```
python3 robot_red_neuronal_memoria.py \\  
--semilla 7 --generaciones 40 --sin-pausa
```

La animacion del mejor individuo aparece despues de la evolucion. Cada cuadro muestra la observacion binaria, la accion elegida y la posicion actual.

7. Guardar y cargar el mejor individuo

Para entrenar y guardar la red:

```
python3 robot_red_neuronal_memoria.py \\  
--generaciones 40 --guardar-mejor mejor_red.pt
```

El archivo `mejor_red.pt` contiene los pesos de PyTorch y los metadatos basicos de la red. Para cargarlo sin evolucionar otra vez:

```
python3 robot_red_neuronal_memoria.py \\  
--cargar-red mejor_red.pt
```

Guardar la red separa dos fases del trabajo: primero se aprende una politica y despues se prueba esa misma politica bajo nuevas condiciones.

8. Probar en un mapa distinto

1. Ejecuta una evolucion y guarda `mejor_red.pt`.
2. Cambia manualmente `OBSTACULOS`, `INICIO` o `META` en el archivo.
3. Conserva el mismo `TAMANO` y evita colocar inicio o meta sobre un obstaculo.
4. Ejecuta con `-cargar-red mejor_red.pt`.
5. Registra si la red llega, cuantos pasos usa, cuantos choques produce y si entra en un ciclo.

Para que la comparacion sea valida, no se deben cambiar los pesos entre las dos pruebas. Solo se modifica el mapa.

9. Objetivo experimental

En este laboratorio se propone responder preguntas como:

- ¿La memoria sensorial reduce la cantidad de ciclos?
- ¿La red aprende a escapar de patrones repetitivos?
- ¿La red con memoria llega a la meta en mapas mas complejos?
- ¿Que pasa si el fitness penaliza la repeticion de posiciones y secuencias?

10. Conclusiones

La clave pedagogica de este laboratorio es que la inteligencia no surge solo de ver el entorno, sino tambien de recordar lo que se vio y lo que se hizo. La modificacion principal no es redefinir completamente el problema, sino extender la representacion del estado para que el robot tenga memoria temporal. De ese modo se conserva la estructura de los laboratorios previos y se introduce una mejora concreta para reducir ciclos sensoriomotores.

11. Diseño del ejercicio

La idea es que ustedes trabajen sobre la base ya entregada y realicen cambios puntuales, sin reconstruir todo el proyecto desde cero. Se sugiere:

1. Revisar el codigo de los laboratorios previos y la base del laboratorio 8 entregada para identificar donde se define la observacion de la red y como se toma la accion.
2. Modificar la entrada de la red para incluir s_{t-1} y a_{t-1} .
3. Mantener el algoritmo genetico y la funcion de fitness, ajustando solo lo necesario para penalizar bucles.

4. Entrenar una poblacion de redes con memoria sensorial.
5. Guardar la mejor red.
6. Cambiar el mapa o la posicion inicial.
7. Probar la misma red sin volver a evolucionar.
8. Comparar exito, choques, pasos y ciclos.

12. Metrica principal

Se puede mantener el fitness anterior y agregar una penalizacion por bucles:

$$F = 500 - 35d + 4p - 30c - 3r + 2000m - 10b$$

donde: - d es la distancia a la meta, - p son pasos utiles, - c choques, - r repeticion de posiciones, - m indica si se alcanzo la meta, - b penaliza bucles sensorimotores.

13. Entrega esperada

Cada grupo debe entregar:

- una version modificada del codigo base de laboratorio anterior,
- comentarios claros sobre que partes se conservaron y que partes se cambiaron,
- scripts de entrenamiento y evaluacion,
- tabla de resultados,
- analisis de los ciclos,
- conclusion sobre la importancia de la memoria temporal.

14. Conclusiones

La clave pedagogica de este laboratorio es que la inteligencia no surge solo de ver el entorno, sino tambien de recordar lo que se vio y lo que se hizo. La tarea no es partir de cero, sino usar la base ya aprendida en los laboratorios anteriores, reconocer el punto de extension y modificar la politica para incorporar memoria temporal. Eso permite que el robot reaccione mejor a secuencias repetitivas y a la aparicion de bucles.